ps-vi leste mye på nettet på ulike nttsider (ingen spesifikke), bare om tips og hvilek commands man kan bruke

ps- også bruke vi KI til å bare få hjelpt hvordan man lasta nye versjon av jupyter og pyhton fra terminalen, og at siden jeg har ac så sa den i generate_data-for_d2 filen at jeg måtte endre linje 11 til det som står der nå slikt mac-en min kunne lese den, fordi vi fikk meldiger at pc min ikke fant filen osv

ellers er dette loggen:

hva er en virtual enviorment i git eller vs code

Et virtual environment (virtuelt miljø) i Python er et eget, isolert miljø for et prosjekt, der du kan installere biblioteker og pakker uten at de påvirker andre prosjekter på PC-en din.

Det brukes ofte sammen med VS Code og Git, men de har forskjellige roller.

1. Hva er et virtual environment?

Se for deg at du jobber med to Python-prosjekter:

- Prosjekt A trenger `numpy` versjon 1.
- Prosjekt B trenger `numpy` versjon 2.

Uten virtuelle miljøer kan det oppstå konflikter mellom versjonene. Med virtuelle miljøer har hvert prosjekt sine egne pakker.

PC-en din

Python installert på systemet

Prosjekt A

Virtual environment A

numpy 1.x

pandas

Prosjekt B

Virtual environment B

numpy 2.x

flask

2. Hva er forskjellen på Git og VS Code?

- VS Code er programmet du skriver og kjører koden i.
- Git holder styr på endringer i koden, slik at du kan gå tilbake til tidligere versjoner og samarbeide med andre.
- Virtual environment (`venv`) isolerer Python-pakkene som et prosjekt bruker.

De fungerer altså sammen, men gjør forskjellige ting.

3. Hvordan lager du et virtual environment i VS Code?

Hvis du bruker Python, kan du gjøre dette i terminalen i VS Code.

Steg 1: Åpne prosjektmappen i VS Code.

Steg 2: Åpne terminalen via `Terminal → New Terminal`.

Steg 3: Opprett et virtuelt miljø.

På Windows:

bash
```
python -m venv .venv
```

På Mac/Linux:

bash
```
python3 -m venv .venv
```

Dette oppretter en mappe som heter `.venv` i prosjektet ditt.

Steg 4: Aktiver miljøet.

Windows PowerShell:

powershell
```
.\.venv\Scripts\Activate.ps1
```

Mac/Linux:

bash
```
source .venv/bin/activate
```

Når det er aktivert, vil du ofte se `(.venv)` foran teksten i terminalen.

Steg 5: Installer pakkene du trenger.

bash
```
pip install numpy pandas
```

Nå installeres pakkene i det virtuelle miljøet i stedet for i det vanlige Python-miljøet ditt.

Steg 6: Velg riktig Python i VS Code.

Trykk `Ctrl + Shift + P` på Windows/Linux eller `Cmd + Shift + P` på Mac. Søk etter `Python: Select Interpreter`, og velg Python-versjonen som ligger i `.venv`.

4. Hva med Git?

Hvis du bruker Git, er det viktig å vite at du vanligvis ikke laster opp selve `.venv`-mappen til GitHub. Den kan være stor og er knyttet til miljøet på din egen PC.

I stedet kan du lagre pakkelisten:

bash
```
pip freeze > requirements.txt
```

Andre kan da opprette sitt eget miljø og installere de samme pakkene med:

bash
```
pip install -r requirements.txt
```

Legg gjerne dette i `.gitignore`:

gitignore
```
.venv/
```

Da ignorerer Git miljømappen.

Kort oppsummert: VS Code er arbeidsplassen din, Git holder styr på kodeendringer, og et virtual environment sørger for at hvert Python-prosjekt har sine egne avhengigheter.

Vil du at jeg skal vise deg et helt konkret eksempel på hvordan du bruker VS Code, Git og et virtual environment sammen i ett Python-prosjekt?

hva er re bibioteket og hva brukes dte til

Python sitt `re`-bibliotek brukes til å søke etter, finne, kontrollere og endre tekst ved hjelp av noe som heter regular expressions (regulære uttrykk).

Det er et innebygd bibliotek i Python, så du trenger ikke å installere det med `pip`.

1. Hva brukes `re` til?

Du kan for eksempel bruke det til å:

- 🔎 Finne tekst: Finne alle tallene i en tekst.
- ✅ Validere tekst: Sjekke om en tekst følger et bestemt mønster, for eksempel et telefonnummer.
- ✂️ Erstatte tekst: Bytte ut bestemte ord eller tegn.
- 📧 Finne e-postadresser: Søke etter tekst som ligner på e-postadresser.
- 🧹 Rydde tekst: Fjerne unødvendige tegn eller mellomrom.

2. Eksempel på hvordan det fungerer

La oss si at du har denne teksten:

python
```
import re

tekst = "Jeg har 3 epler og 12 bananer."

tall = re.findall(r"\d+", tekst)

print(tall)
```

Resultatet blir:

text
```
['3', '12']
```

Hvordan fungerer det?

- `import re` importerer biblioteket.
- `re.findall()` finner alle treff på et mønster.
- `r"\d+"` betyr at Python skal finne sekvenser med ett eller flere sifre.
- `tekst` er teksten vi søker i.

Altså finner programmet alle tallene i setningen.

3. De viktigste funksjonene i `re`

Funksjon	Hva den gjør
`re.findall()`	Finner alle treff
`re.search()`	Finner det første treffet
`re.match()`	Sjekker om mønsteret finnes fra starten av teksten
`re.sub()`	Erstatter treff med annen tekst
`re.split()`	Deler tekst basert på et mønster

Et eksempel på å erstatte tall med `X`:

python

```
import re

tekst = "Jeg har 3 epler og 12 bananer."

ny_tekst = re.sub(r"\d+", "X", tekst)

print(ny_tekst)
```

Resultat:

text
```
Jeg har X epler og X bananer.
```

4. Hva betyr symbolene i `re`?

Disse symbolene brukes til å beskrive mønstre:

Symbol	Betydning	Eksempel
`\d`	Et siffer fra 0 til 9	`\d` finner `5`
`\w`	Et bokstav-/siffertegn eller _	`\w+` finner ordlignende sekvenser
`\s`	Et mellomrom eller annet whitespace-tegn	`\s+` finner ett eller flere
`+`	Ett eller flere	`\d+` finner `123`
`*`	Null eller flere	`\d*`
`.`	Et hvilket som helst tegn, vanligvis unntatt linjeskift	`a.c` kan finne `abc`
`^`	Starten av teksten	`^Hei`
`$`	Slutten av teksten	`slutt$`

5. Et praktisk eksempel

Hvis du vil finne e-postadresser i en tekst:

python
```
import re

tekst = "Kontakt oss på hei@example.com eller test@gmail.com"

eposter = re.findall(
    r"[\w.+-]+@[\w.-]+\.[a-zA-Z]{2,}",
    tekst
)

print(eposter)
```

Resultat:

text
```
['hei@example.com', 'test@gmail.com']
```

Mønsteret finner vanlige e-postadresser, men er ikke en fullstendig validering av alle gyldige e-postadresser.

Kort sagt: `re` er nyttig når du vil jobbe med tekst på en systematisk måte, spesielt når du leter etter bestemte mønstre i store mengder tekst.

hva er alle sånn symboler eller \s osv

Når du bruker `re` i Python, finnes det mange symboler som har spesielle betydninger. Disse brukes i regex (regular expressions) for å søke etter bestemte mønstre i tekst.

Her er en oversikt over de viktigste symbolene, med eksempler du kan teste i VS Code.

1. Spesialtegn som `\d`, `\s` og `\w`

Disse er noen av de mest brukte:

Symbol	Betydning	Eksempel
`\d`	Et siffer (0–9)	`\d` finner `7`
`\D`	Alt som ikke er et siffer	`\D` finner `a`
`\s`	Whitespace, f.eks. mellomrom, tab og linjeskift	`\s` finner et mellomrom
`\S`	Alt som ikke er whitespace	`\S` finner `Hei`
`\w`	Bokstav, siffer eller _	`\w+` finner `Hei123`
`\W`	Alt som ikke er `\w`	`\W` finner `!`
`\b`	Grense mellom et ord og et ikke-ord-tegn	`\bhei\b` finner ordet `hei`
`\B`	En posisjon som ikke er en ordgrense	`\Bhei` finner `hei` inne i et større ord

Husk: Stor bokstav betyr ofte det motsatte av liten bokstav. For eksempel betyr `\d` siffer, mens `\D` betyr ikke-siffer.

2. Symboler som bestemmer antall

Disse forteller hvor mange ganger et tegn eller mønster kan forekomme.

Symbol	Betydning	Eksempel	Treffer
`+`	Én eller flere	`\d+`	`123`
`*`	Null eller flere	`\d*`	`123` eller tom tekst
`?`	Null eller én	`colou?r`	`color`, `colour`
`{3}`	Nøyaktig 3 ganger	`\d{3}`	`123`
`{2,4}`	Mellom 2 og 4 ganger	`\d{2,4}`	`12`, `123`, `1234`
`{2,}`	Minst 2 ganger	`\d{2,}`	`12`, `12345`

Eksempel:

python
```
import re

tekst = "Jeg har 7, 42 og 123 epler."

print(re.findall(r"\d+", tekst))
```

Resultat:

text
```
['7', '42', '123']
```

`\d+` betyr altså: Finn én eller flere sifre etter hverandre.

3. Symboler som bestemmer hvor i teksten du søker

Symbol	Betydning	Eksempel
`^`	Starten av teksten	`^Hei`
`$`	Slutten av teksten	`slutt$`
`.`	Et hvilket som helst tegn, vanligvis unntatt linjeskift	`h.i`
`\`	Brukes blant annet til å gi spesialtegn en annen betydning	`\.` finner et faktisk punktum

Eksempel:

python
```
import re

tekst = "Hei verden"

print(re.findall(r"^Hei", tekst))
print(re.findall(r"verden$", tekst))
```

Resultat:

text
```
['Hei']
['verden']
```

Her finner `^Hei` bare `Hei` i starten, mens `verden$` finner `verden` på slutten.

Viktig: `.` betyr ikke et faktisk punktum i regex. Hvis du vil finne et ekte punktum, bruker du `\.`.

4. Symboler for valg og grupper

Disse lar deg lage mer avanserte mønstre.

Symbol	Betydning	Eksempel
`[]`	Ett tegn fra en gruppe	`[abc]` finner `a`, `b` eller `c`

`[^...]`	Ett tegn som ikke er i gruppen	`[^0-9]` finner et ikke-siffer
`-`	Angir et intervall inne i `[]`	`[a-z]` finner en liten engelsk bokstav
`()`	Grupperer et mønster	`(ha)+` finner `ha`, `haha`
`|`	Betyr ELLER	`ja|nei` finner `ja` eller `nei`

Eksempler:

- `[A-Z]` – en stor engelsk bokstav.
- `[0-9]` – ett siffer.
- `[a-zA-Z]` – en engelsk bokstav, liten eller stor.
- `[0-9]{4}` – nøyaktig fire sifre.
- `(hund\|katt)` – enten `hund` eller `katt`.

I Python-kode kan du for eksempel skrive:

python
```
import re

tekst = "Jeg liker hund og katt."

print(re.findall(r"hund|katt", tekst))
```

Resultat:

text
```
['hund', 'katt']
```

5. Et eksempel som kombinerer flere symboler

La oss si at du vil finne norske mobilnumre skrevet med åtte sifre:

python
```
import re

tekst = "Ring 98765432 eller 12345678."

numre = re.findall(r"\b\d{8}\b", tekst)

print(numre)
```

Resultat:

text
```
['98765432', '12345678']
```

Slik leser du mønsteret:

- `\b` – start ved en ordgrense.
- `\d` – finn et siffer.
- `{8}` – krev nøyaktig åtte sifre.
- `\b` – avslutt ved en ordgrense.

Dette finner åttesifrede tall som ligner telefonnumre, men kontrollerer ikke om numrene faktisk er gyldige eller i bruk.

6. Test deg selv

1. Hva betyr `\d`?

Et mellomrom
Et siffer
Et hvilket som helst tegn

2. Hva finner `\d+` i teksten `abc123def45`?

Bare 1
123 og 45
Hele teksten

3. Hva betyr `\s`?

Et siffer
Et bokstavtegn
Whitespace, for eksempel mellomrom

4. Hvordan finner du et faktisk punktum?

`.`
`\.`
`\d`

5. Hva betyr `\w+`?

En eller flere bokstaver, sifre eller `_`
Bare tall
Bare mellomrom

0 av 5 besvart

Prøv på nytt

Mitt tips: Lær først `\d`, `\s`, `\w`, `+`, `*`, `[]`, `^`, `$` og `\.`. Da kan du allerede lage mange nyttige regex-uttrykk i Python.

hav er feilmeldigen til unicode

En Unicode-feilmelding i Python betyr ofte at Python har problemer med å lese eller bruke tegn som æ, ø, å, emoji eller spesielle symboler. Men den nøyaktige feilen avhenger av feilmeldingen du får.

Vanlige Unicode-feil i Python

1. `UnicodeDecodeError`

Python klarer ikke å dekode bytes til tekst med riktig tegnkoding.

python
```
with open("tekst.txt", encoding="utf-8") as fil:
    tekst = fil.read()
```

Hvis filen ikke faktisk er kodet som UTF-8, kan du få denne feilen.

2. `UnicodeEncodeError`

Python klarer ikke å konvertere tekst til den valgte tegnkodingen.

Dette kan for eksempel skje hvis du prøver å skrive ut et tegn som terminalen ikke støtter.

3. `SyntaxError: (unicode error)`

Denne oppstår ofte når Python tolker en bakoverstrek (`\`) som starten på en escape-sekvens.

For eksempel:

python
```
sti = "C:\Users\Ola"
```

Her kan `\U` bli tolket som starten på en Unicode-sekvens, og du kan få en feil som:

text
```
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes ...
```

Løsning: Bruk en rå tekststreng med `r` foran:

python
```
sti = r"C:\Users\Ola"
```

Eller bruk doble bakoverstreker:

python
```
sti = "C:\\Users\\Ola"
```

Hva betyr Unicode egentlig?

Unicode er en standard som gir tegn, som `A`, `æ`, `ø`, `å` og 😊, egne kodepunkter. UTF-8 er en vanlig måte å lagre disse tegnene som bytes på.

Hvis du sender meg den nøyaktige Unicode-feilmeldingen du får i VS Code, kan jeg forklare hva den betyr og hvordan du fikser den.

hav er egt utf-8

UTF-8 er en måte å lagre tekst og tegn på slik at datamaskinen kan forstå dem. Det kalles en tegnkoding (character encoding).

Datamaskiner lagrer informasjon som bits og bytes (tall), ikke direkte som bokstaver. UTF-8 bestemmer hvordan bokstaver og symboler blir representert som bytes.

1. Hvordan fungerer UTF-8?

Tenk at du skriver denne teksten i Python:

python
```
tekst = "Hei på deg!"
```

Du ser bokstaver på skjermen, men datamaskinen lagrer dem som tall representert av bytes.

For eksempel:

Hei

Teksten du skriver

UTF-8-koding

Bokstaver konverteres til bytes

48 65 69

Bytes vist i heksadesimalt format

I UTF-8 blir bokstavene `H`, `e` og `i` lagret som bytene `48`, `65` og `69` når de vises heksadesimalt.

2. Hvorfor er UTF-8 viktig?

Fordi ikke alle tegn er vanlige engelske bokstaver.

Tegn	UTF-8-bytes (heksadesimalt)
A	41
æ	C3 A6

ø	C3 B8
å	C3 A5
😊	F0 9F 98 8A

UTF-8 bruker mellom én og fire bytes per tegn. Dermed kan det representere både engelske bokstaver, norske bokstaver, mange andre skriftsystemer og emoji.

3. Hvordan bruker du UTF-8 i Python?

Hvis du leser en tekstfil, kan du skrive:

python
```
with open("tekst.txt", "r", encoding="utf-8") as fil:
    tekst = fil.read()

print(tekst)
```

Her betyr `encoding="utf-8"` at Python skal tolke innholdet i filen som UTF-8-kodet tekst.

Når du lagrer tekst til en fil, kan du gjøre det slik:

python
```
with open("tekst.txt", "w", encoding="utf-8") as fil:
    fil.write("Hei, jeg liker æ, ø og å!")
```

Dette bidrar til at norske bokstaver leses og lagres riktig, forutsatt at filen faktisk håndteres som UTF-8.

4. Hva skjer hvis tegnkodingen er feil?

Du kan få tekst som ser slik ut:

- `blåbær` blir til `blÃ¥bÃ¦r`
- Eller Python kan gi en `UnicodeDecodeError`.

Dette skjer gjerne når bytes som er kodet med én tegnkoding, blir tolket som om de brukte en annen.

Kort oppsummert:

- Unicode er standarden som definerer kodepunkter for tegn.
- UTF-8 er en måte å kode disse tegnene som bytes på.
- `encoding="utf-8"` forteller Python hvordan den skal tolke eller lagre tekst i en fil.

Du trenger vanligvis ikke å tenke så mye på dette før du leser filer, behandler tekst eller møter Unicode-feil i Python.

hvordan kanman bruke compile og og multilien i re

I Python sitt `re`-bibliotek kan du bruke `re.compile()` og flagget `re.MULTILINE` for å jobbe mer effektivt med regulære uttrykk, spesielt når du søker gjennom tekst som består av flere linjer.

1. Hva gjør `re.compile()`?

`re.compile()` lager et regex-mønster som du kan lagre i en variabel og bruke flere ganger.

Uten `compile`:

python
```
import re

tekst = "Hei 123"

print(re.findall(r"\d+", tekst))
print(re.search(r"\d+", tekst))
```

Med `compile`:

python
```
import re

mønster = re.compile(r"\d+")

tekst1 = "Hei 123"
tekst2 = "Jeg er 25 år"

print(mønster.findall(tekst1))
print(mønster.findall(tekst2))
```

Resultat:

text
```
['123']
['25']
```

Her betyr `\d+` at du søker etter én eller flere sifre.

Hvorfor bruke `compile`?

- Du kan gjenbruke samme mønster flere ganger.
- Koden blir ryddigere.
- Det kan være mer effektivt når du bruker det samme mønsteret gjentatte ganger.

2. Hva gjør `re.MULTILINE`?

Vanligvis betyr `^` starten av hele teksten, og `$` betyr slutten av hele teksten.

Med `re.MULTILINE` kan `^` og `$` i stedet matche starten og slutten av hver enkelt linje.

Se dette eksemplet:

python
```
import re

tekst = """Hei verden
Hallo Norge
Hei Python"""

mønster = re.compile(r"^Hei", re.MULTILINE)

print(mønster.findall(tekst))
```

Resultat:

text
```
['Hei', 'Hei']
```

Hvorfor?

- Linje 1 begynner med `Hei` → treff.
- Linje 2 begynner med `Hallo` → ikke treff.
- Linje 3 begynner med `Hei` → treff.

Flagget `re.MULTILINE` gjør altså at mønsteret sjekker begynnelsen av hver linje.

3. Hvordan bruker du `compile` og `MULTILINE` sammen?

Du kan sende `re.MULTILINE` som argument til `re.compile()`:

python
```
import re

tekst = """Navn: Ola
Alder: 20
Navn: Kari
Alder: 25"""

mønster = re.compile(r"^Navn: (.+)$", re.MULTILINE)

navn = mønster.findall(tekst)

print(navn)
```

Resultat:

text
```
['Ola', 'Kari']
```

Slik fungerer mønsteret:

Del	Betydning
`^`	Starten av en linje

`Navn:`	Den bokstavelige teksten `Navn:`
`(.+)`	Fang ett eller flere tegn
`$`	Slutten av en linje
`re.MULTILINE`	La `^` og `$` gjelde for hver linje

`findall()` returnerer innholdet i parentesen, altså navnene, i stedet for hele linjene.

4. Forskjellen uten og med `MULTILINE`

python
```
import re

tekst = """Hei
Hallo
Hei igjen"""

print(re.findall(r"^Hei", tekst))
print(re.findall(r"^Hei", tekst, re.MULTILINE))
```

Resultat:

text
```
['Hei']
['Hei', 'Hei']
```

- Uten `MULTILINE`: `^` matcher bare starten av hele teksten.
- Med `MULTILINE`: `^` matcher starten av hver linje.

5. Bonus: Du kan kombinere flere flagg

Du kan også kombinere `re.MULTILINE` med `re.IGNORECASE`, som gjør søket uavhengig av store og små bokstaver.

python
```
import re

tekst = """hei
HEI
Hei"""

mønster = re.compile(r"^hei$", re.MULTILINE | re.IGNORECASE)

print(mønster.findall(tekst))
```

Resultat:

text
```
['hei', 'HEI', 'Hei']
```

Her betyr `|` at flaggene kombineres.

Huskeregel:

- `re.compile()` = lagre et regex-mønster for gjenbruk.
- `re.MULTILINE` = la `^` og `$` fungere ved hver linjegrense.
- `re.IGNORECASE` = ignorer forskjellen mellom store og små bokstaver.

Et viktig tillegg: `MULTILINE` gjør ikke automatisk at `.` matcher linjeskift. Det er flagget `re.DOTALL` som endrer den oppførselen.

forklar dette enklere og hvordan kan man bruke denne og vite man kan bruke dennei koden sin re.finditer() is a function in Python's re module that searches a string for all non-overlapping matches of a regular expression pattern and returns an iterator yielding match objects

La oss gjøre det veldig enkelt! 😊

1. Hva er `re.finditer()`?

`re.finditer()` brukes når du vil finne alle steder et mønster finnes i en tekst, og få informasjon om hvert treff.

Forskjellen fra `re.findall()` er at `finditer()` gir deg match objects. Disse lar deg blant annet hente ut teksten som ble funnet, og hvor i teksten den befinner seg.

Se for deg denne teksten:

python
```
tekst = "Jeg er 20 år og broren min er 25 år"
```

Du vil finne alle tallene.

python
```
import re

tekst = "Jeg er 20 år og broren min er 25 år"

treff = re.finditer(r"\d+", tekst)

for t in treff:
    print(t.group(), t.start(), t.end())
```

Resultat:

text
```
20 7 9
25 31 33
```

Hva betyr tallene?

- `t.group()` gir deg teksten som ble funnet, for eksempel `20`.
- `t.start()` gir startposisjonen i teksten.
- `t.end()` gir posisjonen rett etter slutten av treffet.

Dermed vet du både hva som ble funnet, og hvor det ble funnet.

2. Forskjellen på `findall()` og `finditer()`

`re.findall()`

Gir deg en liste med treffene.

python
```
re.findall(r"\d+", "Jeg er 20 og 25")
```

Resultat:

python
```
['20', '25']
```

`re.finditer()`

Gir deg treff du kan undersøke ett etter ett.

python
```
for t in re.finditer(r"\d+", "Jeg er 20 og 25"):
    print(t.group(), t.start())
```

Resultat:

text
```
20 7
25 13
```

3. Når bør du bruke `finditer()`?

Bruk `finditer()` når du trenger mer enn bare selve teksten.

For eksempel:

- Finne ord og posisjoner: Hvor i teksten forekommer ordet?
- Analysere tekst: Hvor i en lang tekst forekommer e-postadresser eller tall?
- Erstatte bestemte treff selv: Du vil vite nøyaktig hvilke tegn som skal endres.
- Behandle store tekster: Du kan gå gjennom treffene ett om gangen, i stedet for å lage en liste med alle resultatene først.

Hvis du bare trenger en enkel liste over tall eller ord, er `findall()` ofte enklere.

4. Et praktisk eksempel

Si at du har en tekst med priser og vil finne både prisene og hvor de står:

python
```
import re

tekst = "Kaffe koster 35 kr, og te koster 28 kr."

for treff in re.finditer(r"\d+", tekst):
    print("Pris:", treff.group())
    print("Startposisjon:", treff.start())
```

Resultat:

text
```
Pris: 35
Startposisjon: 14
Pris: 28
Startposisjon: 34
```

Her kan du bruke posisjonene hvis du senere vil markere eller endre prisene i originalteksten.

5. Hvordan vet du hvilken funksjon du skal bruke?

En enkel huskeregel:

Hva vil du gjøre?	Bruk
Finne alle treff som en liste	`re.findall()`
Finne det første treffet	`re.search()`
Finne alle treff og få posisjonene deres	`re.finditer()`
Erstatte treff med ny tekst	`re.sub()`
Sjekke om hele teksten følger et mønster	`re.fullmatch()`

Det viktigste å huske: Hvis du trenger å vite både hva som ble funnet og hvor i teksten det finnes, er `re.finditer()` et godt valg. Hvis du bare trenger ordene eller tallene, holder det ofte med `re.findall()`.

kan man skille ting i pythone med |

Ja! 😄 I Python kan du bruke `|` (pipe-symbolet) til flere ting. Siden du nettopp lærte om `str`, `None` og type hints, er det mulig at du har sett det brukt der.

1. `|` i type hints – enten den ene eller den andre typen

Dette er sannsynligvis det du tenker på!

python
```
def finn_navn() -> str | None:
    return None
```

Her betyr `str | None` at funksjonen kan returnere enten:

- `str` – en tekststreng, for eksempel `"Ola"`.
- `None` – ingen verdi.

Et annet eksempel:

python
```
def behandle(verdi: int | str):
    print(verdi)

behandle(25)
behandle("Hei")
```

Her kan `verdi` være enten et heltall (`int`) eller tekst (`str`).

Denne skrivemåten for type hints støttes fra Python 3.10.

2. `|` betyr ELLER i `re` (regex)

Hvis du bruker `re`-biblioteket, kan `|` skille mellom flere mønstre.

python
```
import re

tekst = "Jeg liker hund og katt"

resultat = re.findall(r"hund|katt", tekst)

print(resultat)
```

Resultat:

text
```
['hund', 'katt']
```

Mønsteret `hund|katt` betyr: Finn `hund` eller `katt`.

Du kan også gruppere alternativer:

python
```
r"(hund|katt) mat"
```

Dette matcher enten `hund mat` eller `katt mat`.

3. `|` som OR i mengder og logikk

I Python kan `|` også bety litt andre ting.

Med heltall: bitvis OR

python
```
print(5 | 3)
```

Resultat:

text
```
7
```

Dette handler om binære tall og er litt mer avansert.

Med mengder (`set`): slå sammen elementer

python
```
a = {"eple", "pære"}
b = {"pære", "banan"}

print(a | b)
```

Resultatet inneholder `eple`, `pære` og `banan`, uten duplikater.

Kort oppsummert

Hvor du bruker `|`	Betydning
`str | None`	Enten tekst eller `None`
`int | str`	Enten heltall eller tekst
`r"hund|katt"`	Match `hund` eller `katt` i regex
`sett_a | sett_b`	Slå sammen mengder
`5 | 3`	Bitvis OR

Husk: Hvis du ser `|` sammen med `str`, `int` eller `None` i en funksjonsdefinisjon, er det vanligvis type hints som angir hvilke typer verdier som er tillatt.